



GlaxoSmithKline - Healthcare & Pharmaceuticals

Medicine Outcome Prediction System

UC-01: Clinical Documentation Case Study

-
- > Machine Learning & Deep Learning Implementation
 - > Drug Efficacy & Adverse Event Prediction
 - > End-to-End Data Pipeline with Model Deployment
 - > Regulatory Compliance (HIPAA, FDA, GDPR)
 - > Real-World Public Data Sources for Pharma ML

Technologies: Python | Pandas | Scikit-Learn | TensorFlow/Keras | XGBoost | SHAP | SQLite

Bootcamp Project - Detailed Implementation Guide for Freshers

Table of Contents

1. About GSK & Project Overview
2. Business Problem Statement
3. Use Cases in Healthcare Prediction
4. What is Medicine Outcome Prediction?
5. Dataset Description & Public Data Sources
6. System Architecture & Data Pipeline
7. Prerequisites & Environment Setup
8. Step 1 - Data Collection & Exploration (EDA)
9. Step 2 - Data Cleaning & Preprocessing
10. Step 3 - Feature Engineering for Clinical Data
11. Step 4 - ML Model: Decision Tree Classifier
12. Step 5 - ML Model: Random Forest Classifier
13. Step 6 - ML Model: XGBoost & Gradient Boosting
14. Step 7 - Deep Learning: Neural Network (Keras)
15. Step 8 - Deep Learning: LSTM for Sequential Data
16. Step 9 - Model Evaluation & Comparison
17. Step 10 - Model Explainability (SHAP & LIME)
18. Step 11 - SQL Analytics for Clinical Data
19. Step 12 - Model Deployment (Flask API)
20. Regulatory & Compliance Considerations
21. Interview Questions & Answers
22. Glossary of Medical & ML Terms

1. About GSK & Project Overview

1.1 Who is GSK?

GlaxoSmithKline (GSK) is one of the world's largest pharmaceutical and healthcare companies, headquartered in London, UK. GSK operates in three major business segments:

- **Pharmaceuticals:** Prescription medicines for respiratory, HIV, oncology, and immuno-inflammation.
- **Vaccines:** Preventive vaccines including flu, shingles, meningitis, and pediatric vaccines.
- **Consumer Healthcare:** Over-the-counter products like Sensodyne, Voltaren, and Panadol.

GSK invests heavily in R&D (over \$6 billion annually) and runs hundreds of clinical trials globally. They are increasingly adopting AI/ML to accelerate drug discovery, predict treatment outcomes, and improve patient safety.

1.2 Project Overview

This project builds a Medicine Outcome Prediction System for GSK. The system uses Machine Learning (ML) and Deep Learning (DL) to predict:

- Whether a medicine will be EFFECTIVE or INEFFECTIVE for a patient
- Whether a patient will experience an ADVERSE DRUG REACTION (ADR)
- The likelihood of a patient ADHERING to their medication regimen
- Risk of HOSPITAL READMISSION after treatment

The system ingests patient demographics, lab results, prescription data, and medical history to make these predictions. This guide walks through every step from data exploration to model deployment, with complete code explanations suitable for freshers.

Why This Project Matters

Pharmaceutical companies like GSK lose billions of dollars each year due to adverse drug reactions, failed clinical trials, and patient non-adherence. A prediction system that can identify at-risk patients BEFORE they experience problems can save lives and reduce costs. This is a real-world use case that companies actively hire for.

2. Business Problem Statement

GSK's clinical and commercial teams face several critical challenges that this prediction system addresses:

2.1 The Problem

Every year, adverse drug reactions (ADRs) cause over 100,000 deaths in the US alone, making it one of the leading causes of death. Pharmaceutical companies spend an average of \$2.6 billion to bring a single drug to market, and nearly 90% of drug candidates fail in clinical trials. Additionally, approximately 50% of patients do not take their medications as prescribed, leading to \$300 billion in avoidable healthcare costs annually.

2.2 Business Objectives

- **Predict Drug Efficacy:** Before prescribing, predict whether a specific medicine will work for a given patient based on their profile. Target: 85%+ accuracy.
- **Predict Adverse Events:** Flag patients who are at high risk of adverse drug reactions. Target: 90%+ recall (catching most true adverse events is critical).
- **Optimize Clinical Trials:** Identify patient sub-groups most likely to respond to a drug, reducing trial size and cost by 20-30%.
- **Improve Patient Adherence:** Predict which patients are likely to stop taking medication early, enabling proactive intervention.

2.3 Success Metrics

The following KPIs will measure project success:

Metric	Target	Importance
Accuracy	>= 85%	Overall correctness
Recall (Sensitivity)	>= 90%	Catch adverse events
Precision	>= 80%	Reduce false alarms
AUC-ROC	>= 0.90	Discrimination ability
F1-Score	>= 0.85	Balance precision/recall

NOTE: In healthcare, Recall is more important than Precision. Missing a true adverse event (False Negative) can be fatal, while a false alarm (False Positive) only causes extra testing.

3. Use Cases in Healthcare Prediction

3.1 Drug Efficacy Prediction

Given a patient's demographics (age, gender, weight), lab results (blood counts, liver enzymes), and medical history (comorbidities, prior treatments), predict whether a specific GSK medicine will be effective.

Example: A 55-year-old male with Type 2 diabetes and mild kidney impairment is prescribed Metformin. The model predicts a 72% probability of treatment success based on his eGFR, HbA1c, and BMI values. Since this is below the 80% threshold, the doctor considers an alternative.

- **Model Type:** Binary classification: Effective (1) vs Ineffective (0)

3.2 Adverse Drug Reaction (ADR) Prediction

Predict the probability that a patient will experience a side effect or adverse reaction when taking a specific medicine. This is critical for patient safety.

Example: A 68-year-old female on blood thinners is prescribed a new anti-inflammatory. The model flags a 45% risk of gastrointestinal bleeding based on her age, concurrent medications, and genetic markers (CYP2C9 variant). The doctor adjusts the dosage.

- **Model Type:** Binary classification with probability score

3.3 Clinical Trial Outcome Prediction

At early stages of a clinical trial (Phase I/II), predict whether the trial will succeed in Phase III based on interim biomarker data, patient demographics, and historical trial outcomes.

Example: A Phase II trial for a respiratory drug shows 60% responder rate in patients with specific biomarker levels. The model predicts 78% probability of Phase III success, helping GSK decide whether to invest \$500M+ in the next phase.

- **Model Type:** Binary classification + survival analysis

3.4 Patient Adherence Prediction

Predict whether a patient will adhere to their medication schedule for the prescribed duration. Non-adherence is a \$300B problem in healthcare.

Example: A patient prescribed a 6-month course of antibiotics for TB. Based on their age, socioeconomic factors, pill burden, and side effect history, the model predicts 35% adherence risk. The care team enrolls them in a medication reminder program.

- **Model Type:** Binary or multi-class (Full/Partial/Non-adherent)

3.5 Hospital Readmission Risk

After discharge, predict whether a patient will be readmitted within 30 days. This helps hospitals and pharma companies track treatment effectiveness.

Example: A heart failure patient discharged after medication adjustment. Model predicts 28% readmission risk based on ejection fraction, medication list, and prior admissions. High-risk patients receive follow-up calls.

- **Model Type:** Binary classification with time-to-event analysis

4. What is Medicine Outcome Prediction?

Medicine Outcome Prediction is a branch of Clinical Decision Support Systems (CDSS) that uses data-driven models to forecast how a patient will respond to a treatment. It sits at the intersection of three fields:

- **Pharmacology:** Understanding how drugs interact with the human body.
- **Machine Learning:** Building predictive models from historical patient data.
- **Pharmacogenomics:** Using genetic information to personalize drug therapy.

4.1 How It Works (Simple Explanation for Freshers)

Imagine you have data on 100,000 patients who took a specific medicine. For each patient, you know:

- Their age, gender, weight, lab results
- What medicine they took and at what dose
- Whether the medicine worked (outcome)

A prediction model learns PATTERNS from this data. For example, it might learn that:

- Patients over 65 with kidney problems respond poorly to Drug X
- Patients with a specific gene variant metabolize Drug Y too fast
- Combining Drug A with Drug B increases adverse reaction risk by 3x

When a NEW patient comes in, the model uses these learned patterns to predict their outcome BEFORE they take the medicine. The doctor can then make a more informed decision.

4.2 ML vs DL for Medicine Prediction

Aspect	Machine Learning	Deep Learning
Data Size	Works with 1K-100K rows	Needs 100K+ rows
Feature Eng.	Manual (domain expert)	Automatic (learns features)
Interpretability	High (SHAP, feature imp.)	Low (black box)
Training Time	Minutes	Hours to days
Best For	Tabular clinical data	Images, sequences, text
Examples	Random Forest, XGBoost	CNN, LSTM, Transformer
Regulatory	Easier to explain to FDA	Harder to get approval

When to Use What?

For tabular patient data (demographics, labs, vitals): Use ML models (Random Forest, XGBoost). For medical images (X-rays, pathology slides): Use Deep Learning (CNN). For time-series data (vitals over time, medication history): Use LSTM/Transformer. In practice, most pharma companies start with ML and add DL for specific sub-problems.

5. Dataset Description & Public Data Sources

5.1 Required Data Schema for Medicine Prediction

A medicine outcome prediction system needs the following categories of data. Below is the ideal schema with example columns:

Patient Demographics

Column	Type	Description
patient_id	INT	Unique patient identifier
age	INT	Age in years
gender	CHAR	M / F / Other
weight_kg	FLOAT	Body weight in kilograms
bmi	FLOAT	Body Mass Index
ethnicity	VARCHAR	Ethnic background (affects drug metabolism)

Lab Results

Column	Type	Description
hemoglobin	FLOAT	Blood hemoglobin level (g/dL)
wbc_count	FLOAT	White blood cell count (cells/uL)
alt_enzyme	FLOAT	Liver enzyme ALT (U/L)
ast_enzyme	FLOAT	Liver enzyme AST (U/L)
creatinine	FLOAT	Kidney function marker (mg/dL)
egfr	FLOAT	Estimated glomerular filtration rate
hba1c	FLOAT	Diabetes marker (%)

Prescription Data

Column	Type	Description
drug_name	VARCHAR	Name of the prescribed medicine
dosage_mg	FLOAT	Dosage in milligrams
duration_days	INT	Prescribed duration
route	VARCHAR	Oral, IV, Topical, etc.
concurrent_drugs	INT	Number of other medications

Target Variable

Column	Type	Description
treatment_outcome	INT	1=Effective, 0=Ineffective

adverse_event	INT	1=ADR occurred, 0=No ADR
---------------	-----	--------------------------

5.2 Public Data Sources (Real & Free)

Since actual GSK client data is proprietary and confidential, here are real publicly available datasets that closely mirror pharmaceutical clinical data:

A. GSK's Own Public Data

- **GSK Clinical Study Register:** <https://www.gsk-studyregister.com> - GSK publishes clinical trial results and anonymized patient-level data for approved medicines. Researchers can request access by submitting a research proposal.
- **ClinicalStudyDataRequest.com:** <https://clinicalstudydatarequest.com> - Multi-sponsor platform (includes GSK, Novartis, Roche) for requesting anonymized patient-level clinical trial data.
- **GSK Data Transparency:** <https://www.gsk.com/en-gb/innovation/trials/data-transparency/> - GSK's initiative providing clinical study reports and summary datasets.

B. Government & Regulatory Datasets (Free, No Approval Needed)

- **FDA FAERS:** <https://open.fda.gov/data/faers/> - FDA Adverse Event Reporting System. Contains millions of real adverse drug reaction reports. Best dataset for ADR prediction. Downloadable as quarterly data files.
- **OpenFDA Drug Labels:** <https://open.fda.gov/apis/drug/> - Drug information, indications, contraindications, and adverse reactions from official drug labels. Free API access.
- **ClinicalTrials.gov:** <https://clinicaltrials.gov> - Full database of 400,000+ clinical trials with metadata, eligibility criteria, outcomes, and results. Downloadable in XML/JSON.

C. Clinical Patient Datasets (Require Credentialing)

- **MIMIC-IV:** <https://physionet.org/content/mimiciv/> - Gold standard ICU dataset from Beth Israel Hospital. Contains ~300,000 admissions with vitals, labs, medications, diagnoses, and outcomes. Requires CITI ethics training (free, takes ~4 hours).
- **eICU:** <https://physionet.org/content/eicu-crd/> - Multi-center ICU data from 200+ hospitals. Similar to MIMIC but with more diversity.

D. Drug & Pharmacogenomics Databases (Free)

- **DrugBank:** <https://go.drugbank.com/> - Comprehensive drug data: chemical properties, targets, interactions, pharmacogenomics. Academic license is free.
- **PharmGKB:** <https://www.pharmgkb.org/> - Gene-drug-disease relationships. Essential for pharmacogenomics-based prediction models.
- **SIDER:** <http://sideeffects.embl.de/> - Side Effect Resource. Maps 1,400+ drugs to their known adverse reactions extracted from drug labels.
- **UCI Drug Review:** <https://archive.ics.uci.edu/ml/datasets/Drug+Review+Dataset> - 200,000+ patient drug reviews with ratings, conditions, and satisfaction. Good for NLP + prediction.

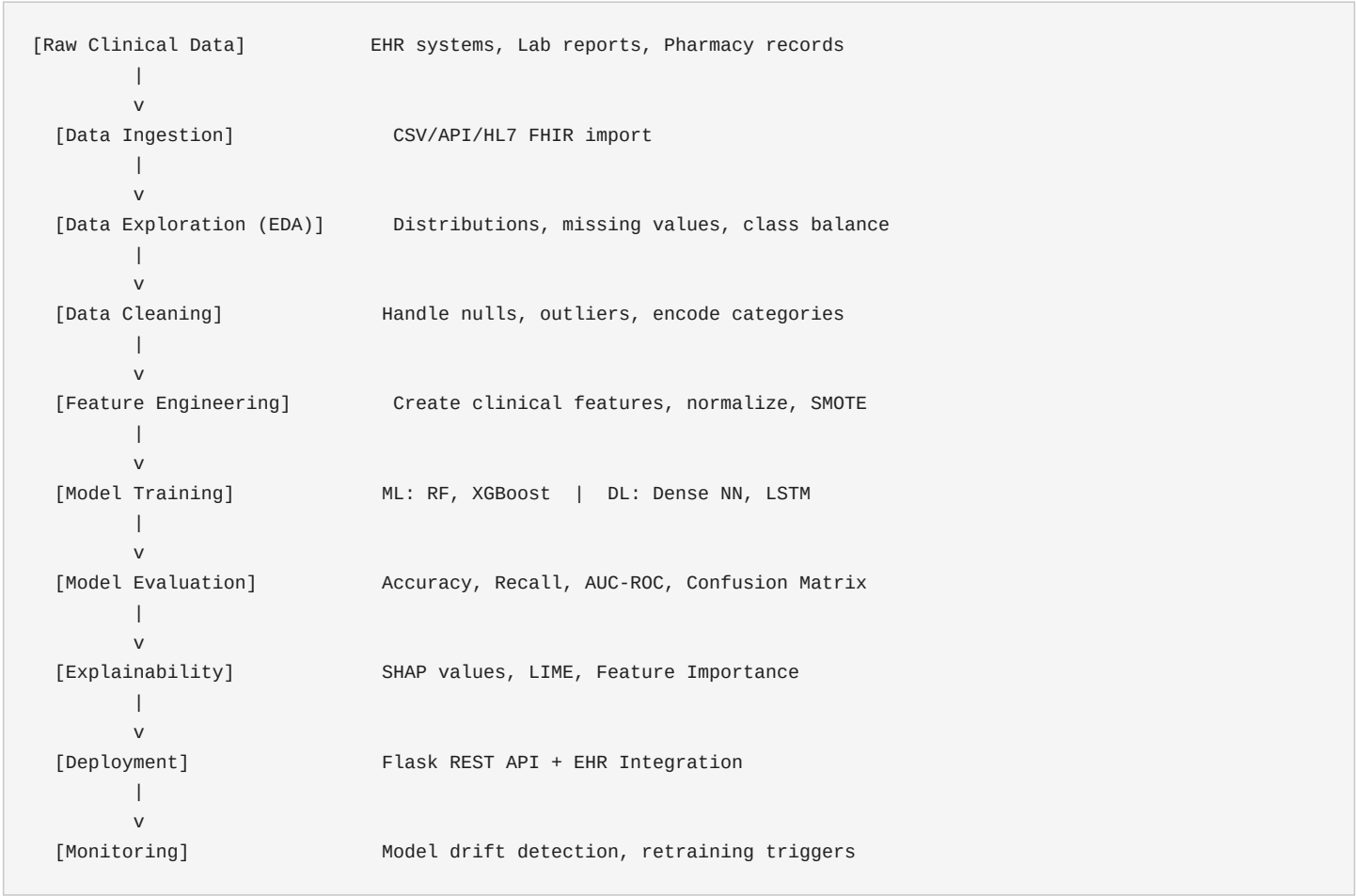
Recommended Quick Start

For an immediate proof-of-concept, download FDA FAERS data (free, no approval). It contains real adverse drug reaction reports with patient demographics, drugs, reactions, and outcomes. You can build an ADR prediction model within hours.

6. System Architecture & Data Pipeline

6.1 End-to-End Architecture

The medicine prediction system follows this pipeline:



6.2 Technology Stack

Layer	Technology	Purpose
Language	Python 3.9+	Core programming language
Data	Pandas, NumPy	Data manipulation & math
Visualization	Matplotlib, Seaborn	Charts & distributions
ML Models	Scikit-Learn, XGBoost	Traditional ML algorithms
DL Models	TensorFlow / Keras	Neural networks (Dense, LSTM)
Explainability	SHAP, LIME	Model interpretation
Database	SQLite / PostgreSQL	Data storage & SQL analytics
API	Flask / FastAPI	Model serving endpoint
Versioning	MLflow / Git	Model & code versioning

7. Prerequisites & Environment Setup

7.1 What You Need to Know (Freshers)

Before starting this project, you should have a basic understanding of:

- **Python Basics:** Variables, loops, functions, lists, dictionaries. If you can write a for loop and use a dictionary, you are ready.
- **Basic Math:** Mean, median, standard deviation, percentages. No advanced math needed for the ML part.
- **What is a Table:** If you understand Excel spreadsheets (rows = patients, columns = features), you understand DataFrames.

Everything else (ML algorithms, neural networks, SHAP) is explained step-by-step in this guide.

7.2 Installation

Step 1: Install Python

Download Python 3.9 or higher from <https://www.python.org/downloads/> and install it. Make sure to check 'Add Python to PATH' during installation.

Step 2: Create a Virtual Environment

```
# Open terminal / command prompt
python -m venv gsk_env

# Activate it
# Windows:
gsk_env\Scripts\activate
# Mac/Linux:
source gsk_env/bin/activate
```

Step 3: Install Required Packages

```
pip install pandas numpy matplotlib seaborn scikit-learn xgboost
pip install tensorflow keras shap lime
pip install flask imbalanced-learn sqlalchemy fpdf2
pip install joblib
```

Step 4: Verify Installation

```
python -c "import pandas; import sklearn; import tensorflow; print('All installed!')"
```

8. Step 1 - Data Collection & Exploration (EDA)

8.1 What is EDA?

Exploratory Data Analysis (EDA) is the FIRST thing you do with any dataset. Think of it as a medical check-up for your data. Before building any model, you need to understand:

- How many patients (rows) and features (columns) do you have?
- What does each column represent?
- Are there missing values? How many?
- Is the target variable balanced (equal number of positive/negative outcomes)?
- What is the distribution of each feature?

8.2 Complete Code with Explanation

```
"""
Step 1 - Exploratory Data Analysis for Clinical Data
Loads patient data, prints statistics, checks for issues.
"""
import pandas as pd
import matplotlib
matplotlib.use('Agg') # Non-interactive backend for saving plots
import matplotlib.pyplot as plt
import seaborn as sns
import os

# ---- Configuration ----
DATA_PATH = 'clinical_data.csv'
OUTPUT_DIR = 'outputs'
TARGET = 'treatment_outcome' # 1=Effective, 0=Ineffective
os.makedirs(OUTPUT_DIR, exist_ok=True)

# ---- Load Data ----
df = pd.read_csv(DATA_PATH)
print('Dataset Shape:', df.shape)
# Output: (50000, 15) means 50K patients, 15 features
```

LINE-BY-LINE EXPLANATION:

- `pd.read_csv()`: Reads the CSV file into a DataFrame (like an Excel table in Python)
- `df.shape`: Returns (rows, columns) - tells you how big the data is
- The TARGET variable is what we want to predict (treatment outcome)

```
# ---- Basic Statistics ----
print('\nColumn Types:')
print(df.dtypes)
# Shows: age=int, weight_kg=float, drug_name=object (text)

print('\nFirst 5 Rows:')
print(df.head())
# Shows a sample of the data so you can visually inspect it

print('\nDescriptive Statistics:')
print(df.describe())
# Shows mean, std, min, max for each numeric column
# Example: mean age = 52, mean BMI = 27.3

print('\nMissing Values:')
print(df.isnull().sum())
# Shows how many blank cells per column
# Example: creatinine has 1,200 missing values
```

WHY THIS MATTERS:

- dtypes tells you if columns are numeric or text (affects preprocessing)
- describe() reveals outliers (e.g., age=999 is clearly wrong)
- Missing values must be handled before training any model

```
# ---- Target Variable Balance ----
print('\nTarget Distribution:')
print(df[TARGET].value_counts())
print(df[TARGET].value_counts(normalize=True))
# Output:
# 1 (Effective)      35000   (70%)
# 0 (Ineffective)   15000   (30%)
# This is IMBALANCED - we need to handle this later
```

NOTE: Class imbalance is very common in healthcare. For adverse events, the ratio can be 95:5 (only 5% of patients have an ADR). Models trained on imbalanced data tend to predict the majority class always. We fix this using SMOTE or class weights in Step 3.

```
# ---- Distribution Plots ----
numeric_cols = df.select_dtypes(include='number').columns.tolist()
numeric_cols = [c for c in numeric_cols if c != 'patient_id']

fig, axes = plt.subplots(3, 3, figsize=(15, 12))
for ax, col in zip(axes.flatten(), numeric_cols[:9]):
    sns.histplot(df[col], kde=True, ax=ax, color='teal')
    ax.set_title(f'Distribution of {col}')
plt.tight_layout()
plt.savefig(f'{OUTPUT_DIR}/step1_clinical_distributions.png', dpi=150)
print('Saved distribution plots')

# ---- Correlation Heatmap ----
plt.figure(figsize=(12, 10))
corr = df[numeric_cols].corr()
sns.heatmap(corr, annot=True, cmap='RdYlGn', center=0, fmt='.2f')
plt.title('Feature Correlation Matrix')
plt.tight_layout()
plt.savefig(f'{OUTPUT_DIR}/step1_correlation_matrix.png', dpi=150)
print('Saved correlation heatmap')
```

WHAT TO LOOK FOR IN EDA:

1. Age distribution: Is it skewed? Are there extreme values like 0 or 150?
2. Lab values: Are they within normal clinical ranges? (e.g., Creatinine 0.6-1.2 mg/dL)
3. Correlation: If two features are highly correlated (>0.9), you may drop one
4. Target balance: If one class dominates, you need SMOTE or class weights

9. Step 2 - Data Cleaning & Preprocessing

9.1 Why Data Cleaning Matters

Real clinical data is MESSY. Hospitals use different systems, lab values can be missing, entries can be duplicated, and there are always outliers (like a recorded weight of 0 kg or age of 200). The famous saying in data science: 'Garbage in, garbage out.' If you train a model on dirty data, the predictions will be unreliable.

9.2 Complete Cleaning Pipeline

Step 2a: Handle Missing Values

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler

df = pd.read_csv('clinical_data.csv')

# Check missing values per column
print('Missing values before cleaning:')
print(df.isnull().sum())

# Strategy:
# - Numeric columns: fill with MEDIAN (not mean, because mean is affected by outliers)
# - Categorical columns: fill with MODE (most frequent value)
# - If >50% missing: DROP the column

# Drop columns with >50% missing
threshold = len(df) * 0.5
df = df.dropna(thresh=threshold, axis=1)

# Fill numeric columns with median
numeric_cols = df.select_dtypes(include='number').columns
for col in numeric_cols:
    if df[col].isnull().sum() > 0:
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f' Filled {col} with median={median_val:.2f}')

# Fill categorical columns with mode
cat_cols = df.select_dtypes(include='object').columns
for col in cat_cols:
    if df[col].isnull().sum() > 0:
        mode_val = df[col].mode()[0]
        df[col].fillna(mode_val, inplace=True)
        print(f' Filled {col} with mode={mode_val}')
```

WHY MEDIAN AND NOT MEAN?

If 100 patients have creatinine values [0.8, 0.9, 1.0, 1.1, 1.2, 15.0], the mean is 3.3 (pulled up by the outlier 15.0), but the median is 1.05 (the true center). In healthcare data, outliers are common (lab errors, data entry mistakes), so median is safer.

Step 2b: Remove Duplicates

```
# Check for duplicate patient records
dups = df.duplicated().sum()
print(f'Duplicate rows: {dups}')

# Remove duplicates, keep first occurrence
df = df.drop_duplicates()
print(f'Shape after removing duplicates: {df.shape}')
```

Step 2c: Handle Outliers (IQR Method)

```
def cap_outliers(df, col):
    """Cap extreme values using IQR method."""
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    before = ((df[col] < lower) | (df[col] > upper)).sum()
    df[col] = df[col].clip(lower, upper)
    print(f' {col}: capped {before} outliers to [{lower:.2f}, {upper:.2f}]')
    return df

# Apply to clinical numeric columns
clinical_cols = ['age', 'bmi', 'dosage_mg', 'hemoglobin',
                 'creatinine', 'alt_enzyme', 'ast_enzyme']
for col in clinical_cols:
    if col in df.columns:
        df = cap_outliers(df, col)
```

IQR METHOD EXPLAINED:

- Q1 = 25th percentile (25% of values are below this)
- Q3 = 75th percentile (75% of values are below this)
- IQR = Q3 - Q1 (the middle 50% range)
- Any value below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$ is an outlier
- We 'cap' (clip) outliers instead of removing them to keep the row

Step 2d: Encode Categorical Variables

```
# Label Encoding for binary/ordinal categories
le = LabelEncoder()
df['gender_encoded'] = le.fit_transform(df['gender'])
# M -> 1, F -> 0

# One-Hot Encoding for non-ordinal categories
df = pd.get_dummies(df, columns=['drug_name', 'route'], drop_first=True)
# Creates: drug_name_Metformin, drug_name_Aspirin, route_IV, etc.

print(f'Shape after encoding: {df.shape}')
print('Columns:', list(df.columns))
```

WHEN TO USE WHICH ENCODING:

- Label Encoding: For binary categories (Male/Female, Yes/No) or ordinal (Low/Medium/High)
- One-Hot Encoding: For non-ordinal categories (drug names, routes) where no order exists
- NEVER use Label Encoding for drug names - the model would think Drug 3 > Drug 1

Step 2e: Normalize Continuous Features

```
# StandardScaler: transforms each feature to mean=0, std=1
scaler = StandardScaler()
scale_cols = ['age', 'bmi', 'dosage_mg', 'hemoglobin',
              'creatinine', 'egfr', 'hba1c']
df[scale_cols] = scaler.fit_transform(df[scale_cols])

# Save cleaned data
df.to_csv('data_cleaned.csv', index=False)
print('Saved cleaned data: data_cleaned.csv')
```

10. Step 3 - Feature Engineering for Clinical Data

10.1 What is Feature Engineering?

Feature engineering is the process of creating NEW columns from EXISTING data to help the model learn better patterns. In healthcare, domain knowledge is crucial - a data scientist who understands clinical concepts will build much better features.

10.2 Clinical Feature Engineering Examples

```

import pandas as pd
import numpy as np

df = pd.read_csv('data_cleaned.csv')

# ---- Feature 1: Kidney Function Category ----
# eGFR (estimated Glomerular Filtration Rate) indicates kidney health
# Clinical staging: >=90 Normal, 60-89 Mild, 30-59 Moderate, <30 Severe
def kidney_stage(egfr):
    if egfr >= 90: return 'Normal'
    elif egfr >= 60: return 'Mild'
    elif egfr >= 30: return 'Moderate'
    else: return 'Severe'

df['kidney_stage'] = df['egfr'].apply(kidney_stage)

# ---- Feature 2: Liver Risk Flag ----
# ALT > 40 OR AST > 40 indicates potential liver issues
df['liver_risk'] = ((df['alt_enzyme'] > 40) | (df['ast_enzyme'] > 40)).astype(int)

# ---- Feature 3: Polypharmacy Flag ----
# Taking 5+ concurrent medications increases ADR risk significantly
df['polypharmacy'] = (df['concurrent_drugs'] >= 5).astype(int)

# ---- Feature 4: BMI Category ----
def bmi_category(bmi):
    if bmi < 18.5: return 'Underweight'
    elif bmi < 25: return 'Normal'
    elif bmi < 30: return 'Overweight'
    else: return 'Obese'

df['bmi_category'] = df['bmi'].apply(bmi_category)

# ---- Feature 5: Age Group ----
df['age_group'] = pd.cut(df['age'], bins=[0, 18, 40, 60, 80, 120],
                        labels=['Pediatric', 'Young Adult', 'Middle Aged',
                                'Senior', 'Elderly'])

# ---- Feature 6: Drug-Age Interaction ----
# Some drugs are more risky for elderly patients
df['elderly_high_dose'] = ((df['age'] > 65) & (df['dosage_mg'] > df['dosage_mg'].median())).astype(int)

# ---- Feature 7: Lab Value Ratios ----
# AST/ALT ratio (De Ritis ratio) > 2 suggests alcoholic liver disease
df['de_ritis_ratio'] = df['ast_enzyme'] / (df['alt_enzyme'] + 0.01) # avoid div by zero

print(f'Features after engineering: {df.shape[1]} columns')
print('New features:', ['kidney_stage', 'liver_risk', 'polypharmacy',
                        'bmi_category', 'age_group', 'elderly_high_dose',
                        'de_ritis_ratio'])

```

Feature Engineering Tips for Healthcare

1. Always consult clinical guidelines for threshold values (e.g., eGFR staging)
2. Interaction features (drug x age, drug x kidney function) are very powerful
3. Ratio features (AST/ALT) carry more clinical meaning than raw values
4. Time-based features (days since last visit, treatment duration) improve predictions
5. Do NOT engineer features from the target variable - that causes data leakage!

11. Step 4 - ML Model: Decision Tree Classifier

11.1 What is a Decision Tree? (Simple Explanation)

A Decision Tree is like a flowchart that a doctor follows to make a diagnosis. At each step, it asks a YES/NO question about a patient's features and branches accordingly.

Example:

Is the patient's age > 65?

YES -> Is their kidney function (eGFR) < 30?

YES -> PREDICT: Drug will be INEFFECTIVE (85% confidence)

NO -> Is their dosage > 500mg?

YES -> PREDICT: High ADVERSE REACTION risk (70%)

NO -> PREDICT: Drug will be EFFECTIVE (80%)

NO -> Is their BMI > 35?

YES -> PREDICT: Moderate efficacy (60%)

NO -> PREDICT: Drug will be EFFECTIVE (90%)

The tree learns these rules automatically from the training data. The algorithm chooses the best questions to ask at each step using a metric called 'Gini Impurity' or 'Information Gain.'

11.2 Key Concepts for Freshers

- **Gini Impurity:** Measures how 'mixed' a group is. If a node has 50% effective and 50% ineffective patients, $Gini=0.5$ (very impure). If 100% effective, $Gini=0$ (pure).
- **Information Gain:** How much 'purity' we gain by splitting on a feature. The algorithm picks the feature that gives the MOST information gain at each step.
- **Max Depth:** How deep the tree can grow. Deeper trees memorize training data (overfitting). Shallow trees are too simple (underfitting). Typical: 5-15.
- **Overfitting:** When the model memorizes the training data but fails on new patients. Like a student who memorizes answers but can't solve new problems.

11.3 Complete Implementation

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, roc_auc_score)
import joblib # for saving the model

# ---- Load cleaned data ----
df = pd.read_csv('data_cleaned.csv')

# ---- Define features (X) and target (y) ----
TARGET = 'treatment_outcome'
drop_cols = [TARGET, 'patient_id'] # don't use these as features
X = df.drop(columns=[c for c in drop_cols if c in df.columns])
y = df[TARGET]

print(f'Features: {X.shape[1]} columns')
print(f'Target: {y.value_counts().to_dict()}')

# ---- Split into training (80%) and testing (20%) ----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# stratify=y ensures both sets have same class ratio
print(f'Train: {X_train.shape[0]} rows, Test: {X_test.shape[0]} rows')

```

WHAT IS train_test_split?

We split the data into two parts:

- Training set (80%): The model **LEARNS** from this data
- Testing set (20%): We **TEST** the model on data it has **NEVER** seen

This simulates real-world usage where the model sees new patients.


```

# ---- Train Decision Tree ----
dt_model = DecisionTreeClassifier(
    max_depth=8,          # Limit tree depth to prevent overfitting
    min_samples_split=20, # Need at least 20 patients to split a node
    min_samples_leaf=10,  # Each leaf must have at least 10 patients
    class_weight='balanced', # Handle class imbalance automatically
    random_state=42
)

dt_model.fit(X_train, y_train) # Training happens here
print('Decision Tree trained successfully!')

# ---- Predict on test set ----
y_pred = dt_model.predict(X_test)
y_prob = dt_model.predict_proba(X_test)[:, 1] # probability of class 1

# ---- Evaluate ----
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(f'AUC-ROC: {roc_auc_score(y_test, y_prob):.4f}')
print('\nClassification Report:')
print(classification_report(y_test, y_pred,
    target_names=['Ineffective', 'Effective']))

# ---- Feature Importance ----
importance = pd.Series(dt_model.feature_importances_,
    index=X.columns).sort_values(ascending=False)
print('\nTop 10 Most Important Features:')
print(importance.head(10))

# ---- Save Model ----
joblib.dump(dt_model, 'outputs/decision_tree_model.pkl')
print('Model saved to outputs/decision_tree_model.pkl')

```

NOTE: `class_weight='balanced'` automatically adjusts for imbalanced classes. If 70% of patients are 'Effective' and 30% are 'Ineffective', it gives 2.3x more weight to 'Ineffective' samples so the model pays more attention to the minority class.

12. Step 5 - ML Model: Random Forest Classifier

12.1 What is Random Forest? (Simple Explanation)

Imagine you are sick and you visit not one doctor, but 100 doctors. Each doctor examines a DIFFERENT subset of your symptoms and gives their diagnosis. You then take the MAJORITY vote as your final diagnosis. That is exactly what Random Forest does.

Random Forest = Many Decision Trees voting together

Why is it better than a single Decision Tree?

- A single tree can make mistakes (overfit to noise in the data)
- When 100 trees vote, individual mistakes get cancelled out
- This is called 'ensemble learning' - the wisdom of the crowd

12.2 How Random Forest Works

- **Step 1 - Bagging:** Create 100+ random subsets of the training data (with replacement). Each subset is slightly different.
- **Step 2 - Random Features:** For each tree, only consider a random subset of features at each split. This ensures trees are diverse (not all identical).
- **Step 3 - Build Trees:** Train one Decision Tree on each random subset.
- **Step 4 - Vote:** For a new patient, each tree gives its prediction. The final prediction is the majority vote (classification) or average (regression).

12.3 Complete Implementation

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, classification_report,
                             roc_auc_score, roc_curve)
import matplotlib.pyplot as plt

# ---- Train Random Forest ----
rf_model = RandomForestClassifier(
    n_estimators=200,      # 200 trees in the forest
    max_depth=12,         # Each tree can go 12 levels deep
    min_samples_split=15, # Need 15 patients to split
    min_samples_leaf=5,   # Each leaf needs 5 patients
    class_weight='balanced',
    random_state=42,
    n_jobs=-1             # Use all CPU cores for speed
)

rf_model.fit(X_train, y_train)
print('Random Forest trained with 200 trees!')

# ---- Predict & Evaluate ----
y_pred_rf = rf_model.predict(X_test)
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]

print(f'Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}')
print(f'AUC-ROC: {roc_auc_score(y_test, y_prob_rf):.4f}')
print('\nClassification Report:')
print(classification_report(y_test, y_pred_rf,
    target_names=['Ineffective', 'Effective']))

# ---- Feature Importance Plot ----
importance_rf = pd.Series(rf_model.feature_importances_,
    index=X.columns).sort_values(ascending=True)

plt.figure(figsize=(10, 8))
importance_rf.tail(15).plot(kind='barh', color='teal')
plt.title('Random Forest - Top 15 Feature Importances')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.savefig('outputs/rf_feature_importance.png', dpi=150)

# ---- Save Model ----
joblib.dump(rf_model, 'outputs/random_forest_model.pkl')

```

13. Step 6 - ML Model: XGBoost & Gradient Boosting

13.1 What is Gradient Boosting? (Simple Explanation)

While Random Forest builds many trees in PARALLEL (all at once), Gradient Boosting builds trees SEQUENTIALLY (one after another). Each new tree focuses on the MISTAKES made by the previous trees.

Analogy: Imagine taking an exam:

- Tree 1: Takes the exam, gets 70% correct
- Tree 2: Studies ONLY the 30% that Tree 1 got wrong, gets 85% of those correct
- Tree 3: Studies the remaining mistakes, gets 90% of those correct
- ... continues until accuracy plateaus

XGBoost (eXtreme Gradient Boosting) is the most popular implementation. It is widely used in healthcare because it typically gives the BEST accuracy on tabular data.

13.2 XGBoost vs Random Forest

Aspect	Random Forest	XGBoost
Tree Building	Parallel (independent)	Sequential (corrective)
Speed	Fast training	Slower but more accurate
Overfitting	Less prone	Can overfit if not tuned
Accuracy	Good	Often best on tabular data
Tuning	Few hyperparameters	Many hyperparameters

13.3 Complete Implementation

```

from xgboost import XGBClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV

# ---- XGBoost Model ----
xgb_model = XGBClassifier(
    n_estimators=200,      # 200 boosting rounds
    max_depth=6,          # Shallower trees (boosting prefers shallow)
    learning_rate=0.1,     # How much each tree contributes
    subsample=0.8,        # Use 80% of data per tree
    colsample_bytree=0.8,  # Use 80% of features per tree
    scale_pos_weight=2.3,  # Handle imbalance (ratio of neg/pos)
    random_state=42,
    eval_metric='auc',
    use_label_encoder=False
)

xgb_model.fit(X_train, y_train)
print('XGBoost trained!')

# ---- Evaluate ----
y_pred_xgb = xgb_model.predict(X_test)
y_prob_xgb = xgb_model.predict_proba(X_test)[:, 1]

print(f'Accuracy: {accuracy_score(y_test, y_pred_xgb):.4f}')
print(f'AUC-ROC: {roc_auc_score(y_test, y_prob_xgb):.4f}')
print(classification_report(y_test, y_pred_xgb,
    target_names=['Ineffective', 'Effective']))

```

Hyperparameter Tuning with GridSearchCV

```

# Grid Search finds the BEST combination of hyperparameters
param_grid = {
    'max_depth': [4, 6, 8],
    'learning_rate': [0.05, 0.1, 0.2],
    'n_estimators': [100, 200, 300],
    'subsample': [0.7, 0.8, 0.9]
}

grid_search = GridSearchCV(
    XGBClassifier(random_state=42, eval_metric='auc', use_label_encoder=False),
    param_grid,
    cv=5,          # 5-fold cross-validation
    scoring='roc_auc',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)
print(f'Best Parameters: {grid_search.best_params_}')
print(f'Best AUC-ROC: {grid_search.best_score_: .4f}')

best_xgb = grid_search.best_estimator_
joblib.dump(best_xgb, 'outputs/xgboost_best_model.pkl')

```

WHAT IS CROSS-VALIDATION?

Instead of splitting data once (train/test), we split it 5 times:

- Fold 1: Train on 80%, test on 20% (part A)
- Fold 2: Train on 80%, test on 20% (part B)
- ... 5 folds total

The final score is the AVERAGE across all 5 folds. This gives a more reliable estimate of model performance.

14. Step 7 - Deep Learning: Neural Network (Keras)

14.1 What is a Neural Network? (Simple Explanation)

A neural network is inspired by how the human brain works. Just like your brain has billions of neurons connected by synapses, a neural network has artificial neurons connected by weights.

Think of it as a pipeline of transformations:

- Input Layer: Receives the patient's features (age, BMI, lab values)
- Hidden Layers: Each layer transforms the data, learning more complex patterns
 - Layer 1 might learn: 'High age + low eGFR = kidney risk'
 - Layer 2 might learn: 'Kidney risk + high dose = ADR risk'
- Output Layer: Gives the final prediction (probability of outcome)

The key difference from Decision Trees: neural networks learn features AUTOMATICALLY. You don't need to tell it to look at age+eGFR - it discovers this pattern itself.

14.2 Key Concepts

- **Neurons:** Each neuron receives inputs, multiplies by weights, adds a bias, and applies an activation function. $\text{output} = \text{activation}(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + \text{bias})$
- **Activation Functions:** ReLU (most common in hidden layers), Sigmoid (output layer for binary classification). $\text{ReLU} = \max(0, x)$ - allows the network to learn non-linear patterns.
- **Epochs:** One pass through the entire training dataset. Typically 50-200 epochs.
- **Batch Size:** Number of samples processed before updating weights. Typical: 32 or 64.
- **Dropout:** Randomly 'turns off' neurons during training to prevent overfitting. Like studying with random pages removed - forces broader learning.
- **Loss Function:** `binary_crossentropy` for binary classification. Measures how wrong the predictions are. Lower = better.

14.3 Complete Implementation

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks
import numpy as np

# ---- Build the Neural Network ----
model = keras.Sequential([
    # Input layer: number of features
    layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    layers.BatchNormalization(), # Normalizes outputs of each layer
    layers.Dropout(0.3),        # Drop 30% of neurons randomly

    # Hidden layer 2
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    # Hidden layer 3
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.2),

    # Output layer: 1 neuron with sigmoid for binary classification
    layers.Dense(1, activation='sigmoid')
])

# ---- Compile: define how the model learns ----
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy', keras.metrics.AUC(name='auc')]
)

model.summary() # Print the architecture

```

ARCHITECTURE EXPLAINED:

- Dense(128): 128 neurons fully connected to all inputs. More neurons = more capacity to learn.
- BatchNormalization: Keeps values in a good range so training is stable and fast.
- Dropout(0.3): Randomly deactivates 30% of neurons per batch. Prevents overfitting.
- Dense(1, sigmoid): Single output neuron. Sigmoid squashes output to 0-1 (probability).


```
# ---- Training Callbacks ----
early_stop = callbacks.EarlyStopping(
    monitor='val_auc',      # Watch validation AUC
    patience=10,            # Stop if no improvement for 10 epochs
    restore_best_weights=True # Keep the best model
)

reduce_lr = callbacks.ReduceLROnPlateau(
    monitor='val_loss',    # Reduce learning rate when loss plateaus
    factor=0.5,            # Halve the learning rate
    patience=5
)

# ---- Handle Class Imbalance ----
from sklearn.utils.class_weight import compute_class_weight
class_weights = compute_class_weight('balanced', classes=np.unique(y_train), y=y_train)
class_weight_dict = dict(enumerate(class_weights))
print(f'Class weights: {class_weight_dict}')
# e.g., {0: 1.67, 1: 0.71} - minority class gets higher weight

# ---- Train the Model ----
history = model.fit(
    X_train, y_train,
    validation_split=0.2,      # Use 20% of training for validation
    epochs=100,
    batch_size=64,
    class_weight=class_weight_dict,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

```
# ---- Evaluate on Test Set ----
y_prob_nn = model.predict(X_test).flatten()
y_pred_nn = (y_prob_nn >= 0.5).astype(int)

print(f'Accuracy: {accuracy_score(y_test, y_pred_nn):.4f}')
print(f'AUC-ROC: {roc_auc_score(y_test, y_prob_nn):.4f}')
print(classification_report(y_test, y_pred_nn,
                             target_names=['Ineffective', 'Effective']))

# ---- Plot Training History ----
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(history.history['loss'], label='Train Loss')
axes[0].plot(history.history['val_loss'], label='Val Loss')
axes[0].set_title('Loss Over Epochs')
axes[0].legend()

axes[1].plot(history.history['auc'], label='Train AUC')
axes[1].plot(history.history['val_auc'], label='Val AUC')
axes[1].set_title('AUC Over Epochs')
axes[1].legend()

plt.tight_layout()
plt.savefig('outputs/nn_training_history.png', dpi=150)

# ---- Save Model ----
model.save('outputs/neural_network_model.keras')
print('Neural network model saved!')
```

15. Step 8 - Deep Learning: LSTM for Sequential Data

15.1 What is LSTM? (Simple Explanation)

LSTM (Long Short-Term Memory) is a special type of neural network designed for SEQUENTIAL data - data where ORDER matters.

In healthcare, sequential data includes:

- A patient's vital signs over time (hourly BP readings for 48 hours)
- Medication history over months (drug A, then drug B, then drug C)
- Lab results across multiple visits (creatinine trending up over 6 months)

Why LSTM instead of a regular neural network?

A regular Dense network treats all inputs independently - it doesn't understand that 'creatinine was 1.0 yesterday, 1.5 today, 2.0 tomorrow' is a WORSENING TREND. LSTM has a 'memory cell' that remembers previous time steps and detects such patterns.

Analogy: A Dense network is like reading random pages of a patient's chart. LSTM reads the chart in chronological order and remembers the patient's journey.

15.2 When to Use LSTM in Healthcare

- **Time-series vitals:** Predicting patient deterioration from hourly ICU monitor data.
- **Medication sequences:** Predicting ADR based on sequence of drugs prescribed over months.
- **Lab trends:** Predicting kidney failure from trending lab values across visits.
- **EHR event sequences:** Predicting readmission from sequence of diagnosis codes.

15.3 Complete Implementation

```

import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

# ---- Prepare Sequential Data ----
# For LSTM, data must be 3D: (samples, timesteps, features)
# Example: 10,000 patients, 12 monthly visits, 8 features per visit

# Reshape tabular data into sequences
# (In real scenario, you would have actual time-series data)
n_timesteps = 12    # 12 monthly observations
n_features = 8       # 8 clinical features per observation

# Create synthetic sequential data for demonstration
X_seq_train = X_train.values.reshape(-1, n_timesteps, n_features)
X_seq_test = X_test.values.reshape(-1, n_timesteps, n_features)
# Note: In production, you'd have actual time-series patient data

```

```

# ---- Build LSTM Model ----
lstm_model = keras.Sequential([
    # LSTM layer: learns temporal patterns
    layers.LSTM(64, return_sequences=True,
                input_shape=(n_timesteps, n_features)),
    layers.Dropout(0.3),

    # Second LSTM layer: learns higher-level temporal patterns
    layers.LSTM(32, return_sequences=False),
    layers.Dropout(0.3),

    # Dense layers for classification
    layers.Dense(16, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(1, activation='sigmoid')
])

lstm_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy', keras.metrics.AUC(name='auc')]
)

lstm_model.summary()

```

LSTM ARCHITECTURE EXPLAINED:

- LSTM(64, return_sequences=True): 64 memory units, outputs sequence for next LSTM layer.
- LSTM(32, return_sequences=False): 32 memory units, outputs single vector (final state).
- return_sequences=True: Pass full sequence to next layer (needed when stacking LSTMs).
- return_sequences=False: Only pass final hidden state (used before Dense layers).
- The rest (Dense, Dropout, Sigmoid) is same as the standard neural network.

```
# ---- Train LSTM ----
history_lstm = lstm_model.fit(
    X_seq_train, y_train_seq,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    class_weight=class_weight_dict,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

# ---- Evaluate ----
y_prob_lstm = lstm_model.predict(X_seq_test).flatten()
y_pred_lstm = (y_prob_lstm >= 0.5).astype(int)

print(f'LSTM Accuracy: {accuracy_score(y_test_seq, y_pred_lstm):.4f}')
print(f'LSTM AUC-ROC: {roc_auc_score(y_test_seq, y_prob_lstm):.4f}')

# ---- Save ----
lstm_model.save('outputs/lstm_model.keras')
```

When NOT to Use LSTM

If your data is purely tabular (one row per patient, no time dimension), LSTM is overkill. Use Random Forest or XGBoost instead - they will be faster, more accurate, and easier to explain. LSTM shines only when you have actual time-series data (vitals over hours, lab trends over months).

16. Step 9 - Model Evaluation & Comparison

16.1 Evaluation Metrics Explained for Freshers

Confusion Matrix

A confusion matrix shows WHAT the model got right and WHAT it got wrong:

	Predicted: Effective	Predicted: Ineffective
Actual: Effective	TRUE POSITIVE (TP)	FALSE NEGATIVE (FN)
Actual: Ineffective	FALSE POSITIVE (FP)	TRUE NEGATIVE (TN)

- TP: Correctly predicted Effective (GOOD)
- TN: Correctly predicted Ineffective (GOOD)
- FP: Predicted Effective but was actually Ineffective (BAD - patient gets wrong drug)
- FN: Predicted Ineffective but was actually Effective (BAD - patient misses a good drug)

Key Metrics

- **Accuracy:** $(TP+TN) / \text{Total}$. Overall correctness. Can be misleading with imbalanced data.
- **Precision:** $TP / (TP+FP)$. Of all patients predicted 'Effective', how many truly were?
- **Recall (Sensitivity):** $TP / (TP+FN)$. Of all truly Effective patients, how many did we catch?
- **F1-Score:** Harmonic mean of Precision and Recall. Balances both.
- **AUC-ROC:** Area Under the ROC Curve. Measures ability to distinguish between classes. 0.5 = random guess, 1.0 = perfect. Good models: >0.85 .
- **AUC-PR:** Area Under the Precision-Recall Curve. Better than AUC-ROC for imbalanced data.

IMPORTANT: In healthcare, Recall for adverse events is the MOST important metric. A False Negative means a patient with an adverse reaction was MISSED - this can be life-threatening. It is better to have some false alarms (lower Precision) than to miss cases.

16.2 Model Comparison Code

```

from sklearn.metrics import (roc_curve, auc, precision_recall_curve,
                             average_precision_score)
import matplotlib.pyplot as plt

# ---- Collect all model results ----
models = {
    'Decision Tree': (y_pred, y_prob),
    'Random Forest': (y_pred_rf, y_prob_rf),
    'XGBoost': (y_pred_xgb, y_prob_xgb),
    'Neural Network': (y_pred_nn, y_prob_nn),
}

# ---- Comparison Table ----
results = []
for name, (pred, prob) in models.items():
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, pred),
        'Precision': precision_score(y_test, pred),
        'Recall': recall_score(y_test, pred),
        'F1': f1_score(y_test, pred),
        'AUC-ROC': roc_auc_score(y_test, prob),
    })

comparison_df = pd.DataFrame(results)
comparison_df = comparison_df.sort_values('AUC-ROC', ascending=False)
print(comparison_df.to_string(index=False))
comparison_df.to_csv('outputs/model_comparison.csv', index=False)

```

```

# ---- ROC Curve Comparison ----
plt.figure(figsize=(10, 8))
for name, (pred, prob) in models.items():
    fpr, tpr, _ = roc_curve(y_test, prob)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, linewidth=2, label=f'{name} (AUC={roc_auc:.3f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random (AUC=0.500)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate (Recall)')
plt.title('ROC Curve - All Models Comparison')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('outputs/roc_curve_comparison.png', dpi=150)
print('Saved ROC comparison plot')

```

17. Step 10 - Model Explainability (SHAP & LIME)

17.1 Why Explainability Matters in Healthcare

In healthcare, a prediction alone is NOT enough. A doctor will NEVER trust a model that says 'this drug will be ineffective' without explaining WHY. Regulators (FDA, EMA) also require model interpretability for clinical decision support systems.

Two main tools:

- SHAP (SHapley Additive exPlanations): Shows how much each feature contributes to a prediction
- LIME (Local Interpretable Model-agnostic Explanations): Approximates the model locally

17.2 SHAP Implementation

```
import shap

# ---- SHAP for XGBoost (or Random Forest) ----
explainer = shap.TreeExplainer(best_xgb) # or rf_model
shap_values = explainer.shap_values(X_test)

# ---- Global Feature Importance (Summary Plot) ----
# Shows which features matter MOST across ALL patients
plt.figure(figsize=(12, 8))
shap.summary_plot(shap_values, X_test, feature_names=X.columns,
                  show=False)
plt.tight_layout()
plt.savefig('outputs/shap_summary.png', dpi=150, bbox_inches='tight')
print('Saved SHAP summary plot')
```

HOW TO READ A SHAP SUMMARY PLOT:

- Each row is a feature (e.g., 'age', 'creatinine')
- Each dot is a patient
- Dot color: RED = high feature value, BLUE = low feature value
- Dot position: RIGHT = pushes prediction toward 'Effective', LEFT = toward 'Ineffective'

Example interpretation:

- Red dots (high creatinine) on the LEFT means high creatinine pushes toward 'Ineffective'
- This makes clinical sense: poor kidney function = poor drug clearance = lower efficacy


```
# ---- Individual Patient Explanation (Force Plot) ----
# Explain prediction for a SINGLE patient (e.g., patient index 0)
patient_idx = 0
print(f'Patient {patient_idx} prediction: {y_prob_xgb[patient_idx]:.3f}')
print(f'Actual outcome: {y_test.iloc[patient_idx]}')

shap.force_plot(
    explainer.expected_value,
    shap_values[patient_idx],
    X_test.iloc[patient_idx],
    feature_names=X.columns,
    matplotlib=True
)
plt.savefig('outputs/shap_force_patient0.png', dpi=150, bbox_inches='tight')
```

FORCE PLOT INTERPRETATION:

Shows how each feature pushes the prediction UP (toward Effective) or DOWN (toward Ineffective) for this specific patient. Red features push right (Effective), blue push left (Ineffective). The model's prediction is where all forces balance out.

17.3 LIME Implementation

```
import lime
import lime.lime_tabular

# ---- Create LIME Explainer ----
lime_explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X.columns.tolist(),
    class_names=['Ineffective', 'Effective'],
    mode='classification'
)

# ---- Explain a single prediction ----
explanation = lime_explainer.explain_instance(
    X_test.iloc[0].values,
    best_xgb.predict_proba,
    num_features=10
)

# Show top contributing features
print('LIME Explanation for Patient 0:')
for feature, weight in explanation.as_list():
    direction = 'Effective' if weight > 0 else 'Ineffective'
    print(f' {feature}: {weight:+.4f} -> pushes toward {direction}')

fig = explanation.as_pyplot_figure()
fig.tight_layout()
fig.savefig('outputs/lime_explanation_patient0.png', dpi=150)
```

18. Step 11 - SQL Analytics for Clinical Data

18.1 Why SQL in a Data Science Project?

SQL (Structured Query Language) is used to query and analyze data stored in databases. In a real GSK project, patient data lives in databases (not CSV files). Data scientists must know SQL to:

- Extract the right patient cohort from millions of records
- Calculate aggregate statistics (average response rate per drug)
- Join data from multiple tables (patients + labs + prescriptions)
- Create reports for clinical teams

18.2 Clinical SQL Queries

```
import sqlite3
import pandas as pd

# ---- Load data into SQLite ----
df = pd.read_csv('data_cleaned.csv')
conn = sqlite3.connect(':memory:') # In-memory database
df.to_sql('patients', conn, index=False, if_exists='replace')

def run_query(title, sql):
    print(f'\n{title}')
    print('=' * 60)
    result = pd.read_sql_query(sql, conn)
    print(result.to_string(index=False))
    return result
```

```
# Query 1: Treatment outcome distribution
run_query('Q1: Overall Treatment Outcomes', '''
    SELECT treatment_outcome,
           COUNT(*) as patient_count,
           ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM patients), 2) as pct
    FROM patients
    GROUP BY treatment_outcome
    ORDER BY treatment_outcome
''')

# Query 2: Average outcome by age group
run_query('Q2: Efficacy by Age Group', '''
    SELECT CASE
           WHEN age < 30 THEN 'Under 30'
           WHEN age BETWEEN 30 AND 50 THEN '30-50'
           WHEN age BETWEEN 51 AND 65 THEN '51-65'
           ELSE 'Over 65'
        END as age_group,
           COUNT(*) as total_patients,
           SUM(treatment_outcome) as effective_count,
           ROUND(AVG(treatment_outcome) * 100, 2) as efficacy_pct
    FROM patients
    GROUP BY age_group
    ORDER BY efficacy_pct DESC
''')

# Query 3: Top drugs by efficacy rate
run_query('Q3: Drug Efficacy Ranking', '''
    SELECT drug_name,
           COUNT(*) as total_prescribed,
           SUM(treatment_outcome) as effective,
           ROUND(AVG(treatment_outcome) * 100, 2) as efficacy_rate
    FROM patients
    GROUP BY drug_name
    HAVING total_prescribed >= 100
    ORDER BY efficacy_rate DESC
    LIMIT 10
''')
```

```
# Query 4: Adverse events by drug and age
run_query('Q4: ADR Risk by Drug and Age', '''
    SELECT drug_name,
           CASE WHEN age > 65 THEN 'Elderly' ELSE 'Non-Elderly' END as age_cat,
           COUNT(*) as patients,
           SUM(adverse_event) as adr_count,
           ROUND(AVG(adverse_event) * 100, 2) as adr_rate
    FROM patients
    GROUP BY drug_name, age_cat
    ORDER BY adr_rate DESC
    LIMIT 15
''')

# Query 5: High-risk patient identification
run_query('Q5: High-Risk Patients (Multiple Risk Factors)', '''
    SELECT patient_id, age, bmi, creatinine, concurrent_drugs,
           treatment_outcome, adverse_event
    FROM patients
    WHERE age > 65
           AND creatinine > 1.5
           AND concurrent_drugs >= 5
    ORDER BY creatinine DESC
    LIMIT 20
''')

# Query 6: Dosage analysis
run_query('Q6: Outcome vs Dosage Level', '''
    SELECT CASE
           WHEN dosage_mg < 100 THEN 'Low (<100mg)'
           WHEN dosage_mg BETWEEN 100 AND 500 THEN 'Medium (100-500mg)'
           ELSE 'High (>500mg)'
    END as dosage_level,
    COUNT(*) as patients,
    ROUND(AVG(treatment_outcome)*100, 2) as efficacy_pct,
    ROUND(AVG(adverse_event)*100, 2) as adr_pct
    FROM patients
    GROUP BY dosage_level
''')
```

19. Step 12 - Model Deployment (Flask API)

19.1 What is Model Deployment?

Building a model in a Jupyter notebook is step 1. Making it available for REAL USE is step 2. Deployment means wrapping your trained model in a web API (Application Programming Interface) so that other systems (hospital software, mobile apps) can send patient data and get predictions.

We use Flask - a lightweight Python web framework - to create a REST API endpoint.

19.2 Flask API Implementation

```

"""
app.py - Flask API for Medicine Outcome Prediction
Run: python app.py
Test: curl -X POST http://localhost:5000/predict -H 'Content-Type: application/json' \
      -d '{"age": 55, "bmi": 28.5, "creatinine": 1.2, ...}'
"""

from flask import Flask, request, jsonify
import joblib
import numpy as np
import pandas as pd

app = Flask(__name__)

# Load the best trained model
model = joblib.load('outputs/xgboost_best_model.pkl')

# Expected feature order (must match training)
FEATURE_COLS = ['age', 'gender_encoded', 'bmi', 'dosage_mg',
                 'hemoglobin', 'creatinine', 'egfr', 'hba1c',
                 'concurrent_drugs', 'liver_risk', 'polypharmacy']

@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.get_json()

        # Validate required fields
        missing = [f for f in FEATURE_COLS if f not in data]
        if missing:
            return jsonify({'error': f'Missing fields: {missing}'}), 400

        # Create feature array in correct order
        features = np.array([[data[f] for f in FEATURE_COLS]])

        # Predict
        probability = model.predict_proba(features)[0][1]
        prediction = int(probability >= 0.5)

        return jsonify({
            'prediction': 'Effective' if prediction == 1 else 'Ineffective',
            'probability': round(float(probability), 4),
            'confidence': 'High' if abs(probability - 0.5) > 0.3 else 'Medium'
                          if abs(probability - 0.5) > 0.15 else 'Low',
            'model': 'XGBoost v1.0',
        })
    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/health', methods=['GET'])
def health():
    return jsonify({'status': 'healthy', 'model_loaded': True})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)

```

Testing the API

```
# Using Python requests library
import requests

patient_data = {
    'age': 55,
    'gender_encoded': 1,
    'bmi': 28.5,
    'dosage_mg': 250,
    'hemoglobin': 13.5,
    'creatinine': 1.2,
    'egfr': 65,
    'hba1c': 7.1,
    'concurrent_drugs': 3,
    'liver_risk': 0,
    'polypharmacy': 0
}

response = requests.post('http://localhost:5000/predict', json=patient_data)
print(response.json())
# Output: {'prediction': 'Effective', 'probability': 0.7834,
#          'confidence': 'High', 'model': 'XGBoost v1.0'}
```

20. Regulatory & Compliance Considerations

Healthcare AI/ML systems are subject to strict regulatory requirements. As a data scientist at GSK, you MUST understand these regulations:

20.1 Key Regulations

- **HIPAA (USA):** Health Insurance Portability and Accountability Act. ALL patient data must be de-identified (remove names, SSN, dates of birth, addresses) before use in ML. Violation penalties: up to \$1.5 million per incident.
- **GDPR (Europe):** General Data Protection Regulation. Patients have the right to know how their data is used, request deletion, and get explanations of automated decisions. Applies to ANY EU citizen's data, even if processed outside the EU.
- **FDA SaMD (USA):** Software as a Medical Device. If your prediction model is used to make CLINICAL DECISIONS (e.g., 'do not prescribe this drug'), it may be classified as a medical device and require FDA clearance (510(k) or De Novo pathway).
- **ICH GCP:** International Council for Harmonisation - Good Clinical Practice. Standards for clinical trial conduct. Any ML model used in trials must follow GCP guidelines.

20.2 Technical Requirements for Compliance

Requirement	Implementation
Data De-identification	Remove/hash all 18 HIPAA identifiers before model training
Audit Trail	Log every prediction: timestamp, model version, inputs, output
Model Versioning	Track model versions with MLflow or DVC. Never overwrite models.
Explainability	SHAP/LIME explanations for every prediction (FDA requirement)
Bias Testing	Test model fairness across age, gender, ethnicity subgroups
Data Encryption	Encrypt data at rest (AES-256) and in transit (TLS 1.2+)
Access Control	Role-based access: only authorized users can access patient data

IMPORTANT: NEVER use real patient names, SSNs, or dates of birth in your model training data. Always use de-identified datasets. Even in this bootcamp project, follow best practices to build good habits for industry work.

21. Interview Questions & Answers

Common interview questions about this project and healthcare ML:

Q1: What is the difference between ML and DL for healthcare?

ML (Machine Learning) uses algorithms like Random Forest and XGBoost that work well on structured/tabular data (patient records, lab values). It requires manual feature engineering but is interpretable. DL (Deep Learning) uses neural networks that automatically learn features and excels at unstructured data (medical images, clinical notes, time-series). DL needs more data and compute but can discover complex patterns. For tabular clinical data, ML typically outperforms DL. For imaging and sequence data, DL wins.

Q2: How do you handle class imbalance in clinical data?

Clinical data is almost always imbalanced (e.g., 5% adverse events). Solutions: 1) SMOTE: synthetically generates minority class samples. 2) Class weights: tell the model to pay more attention to minority class. 3) Threshold tuning: lower the classification threshold from 0.5 to 0.3 for high-recall. 4) Ensemble methods: combine multiple models. I prefer `class_weight='balanced'` in sklearn + threshold tuning for simplicity.

Q3: Why is Recall more important than Precision in ADR prediction?

A False Negative in ADR prediction means a patient with a real adverse reaction was MISSED - they take the drug and suffer harm. A False Positive means a patient was flagged unnecessarily - they get extra monitoring or an alternative drug. The cost of FN (patient harm/death) is far greater than FP (inconvenience). So we optimize for Recall (catching all true ADRs) even if it means some false alarms.

Q4: What is SHAP and why is it needed in healthcare AI?

SHAP (SHapley Additive exPlanations) explains WHY a model made a specific prediction by showing the contribution of each feature. In healthcare, doctors need to understand predictions to trust them and make informed decisions. Regulators (FDA, EMA) also require model interpretability. SHAP tells you: 'This patient was predicted Ineffective because their creatinine is 2.5 (kidney impairment) and they are on 7 concurrent drugs (polypharmacy).'

Q5: How would you deploy this model in a hospital?

1) Wrap the model in a REST API (Flask/FastAPI). 2) Containerize with Docker for consistent deployment. 3) Integrate with the hospital's EHR system via HL7 FHIR APIs. 4) Add authentication (OAuth2) and HIPAA-compliant logging. 5) Set up model monitoring for drift detection. 6) Create a dashboard for clinicians showing predictions + SHAP explanations. 7) Get FDA clearance if used for clinical decisions.

Q6: What is overfitting and how do you prevent it?

Overfitting is when a model memorizes the training data but fails on new data. It's like a student who memorizes exam answers but can't solve new problems. Signs: very high training accuracy but low test accuracy. Prevention: 1) Cross-validation (test on multiple data splits). 2) Regularization (penalize complex models). 3) Dropout (in neural networks, randomly deactivate neurons). 4) Early stopping (stop training when validation performance drops). 5) Limit model complexity (`max_depth`, `min_samples_leaf`).

Q7: What data would you need for a medicine prediction system?

Patient demographics (age, gender, BMI, ethnicity), lab results (blood counts, liver/kidney function, biomarkers), prescription details (drug name, dosage, route, duration), medical history (comorbidities, prior adverse events, allergies), concurrent medications, and optionally genomic data (pharmacogenomic markers like CYP2D6 status). The target variable would be treatment outcome (effective/ineffective) or adverse event occurrence (yes/no).

Q8: Explain the difference between Random Forest and XGBoost.

Both are ensemble methods using multiple Decision Trees. Random Forest builds trees in PARALLEL on random data subsets - each tree is independent. The final prediction is a majority vote. It's robust and hard to overfit. XGBoost builds trees SEQUENTIALLY - each tree corrects the mistakes of the previous one. It optimizes a gradient-based loss function. XGBoost usually achieves higher accuracy but requires more careful hyperparameter tuning. On tabular healthcare data, XGBoost typically wins by 1-3% in AUC-ROC.

Q9: How do you handle missing lab values in clinical data?

Strategy depends on the percentage and reason for missingness: 1) <5% missing: fill with median (numeric) or mode (categorical). 2) 5-30% missing: use iterative imputation (sklearn IterativeImputer) or KNN imputation. 3) >50% missing: drop the column unless clinically critical. 4) Also create a binary 'is_missing' flag column - missingness itself can be informative (e.g., missing lab = test not ordered = doctor didn't suspect that condition). Never use mean for clinical data - outliers skew it. Median is always safer.

Q10: What is the MIMIC dataset and how is it used?

MIMIC (Medical Information Mart for Intensive Care) is a publicly available critical care database from Beth Israel Deaconess Medical Center. MIMIC-IV contains ~300,000 hospital admissions with demographics, vital signs, lab results, medications, procedures, diagnoses, and clinical notes. It requires completing CITI ethics training and signing a data use agreement on PhysioNet. It's the gold standard for healthcare ML research and is used for mortality prediction, readmission prediction, drug interaction analysis, and clinical NLP.

22. Glossary of Medical & ML Terms

ADR	Adverse Drug Reaction - an undesirable effect caused by a medicine
AUC-ROC	Area Under the Receiver Operating Characteristic Curve - measures model discrimination
Bagging	Bootstrap Aggregating - training models on random subsets of data
Batch Size	Number of samples processed before updating model weights in neural networks
BMI	Body Mass Index - $\text{weight(kg)} / \text{height(m)}^2$. Normal: 18.5-24.9
Class Imbalance	When one class has far more samples than another (e.g., 95% healthy, 5% ADR)
Clinical Trial	Research study to test safety/efficacy of a new treatment on patients
CNN	Convolutional Neural Network - DL model for images (X-rays, pathology slides)
Confusion Matrix	Table showing TP, TN, FP, FN - what the model got right and wrong
Cross-Validation	Evaluating model on multiple train/test splits for reliable performance estimates
CYP2D6	Liver enzyme that metabolizes 25% of drugs. Genetic variants affect drug processing
De-identification	Removing patient identifiers (name, SSN, DOB) to protect privacy
Dropout	Randomly deactivating neurons during training to prevent overfitting
EDA	Exploratory Data Analysis - initial investigation of data characteristics
eGFR	Estimated Glomerular Filtration Rate - measures kidney function. Normal: >90
EHR	Electronic Health Record - digital patient medical records
Ensemble	Combining multiple models for better predictions (Random Forest, XGBoost)
Epoch	One complete pass through all training data in neural network training
F1-Score	Harmonic mean of Precision and Recall. Balances both metrics
FAERS	FDA Adverse Event Reporting System - database of drug side effect reports
Feature Engineering	Creating new informative columns from existing data
FHIR	Fast Healthcare Interoperability Resources - standard API for health data exchange
Flask	Lightweight Python web framework for creating APIs
Gini Impurity	Decision Tree metric measuring how mixed a node's classes are
GridSearchCV	Exhaustive search over hyperparameter combinations with cross-validation
HbA1c	Hemoglobin A1c - measures average blood sugar over 3 months. Diabetes: >6.5%
HIPAA	Health Insurance Portability and Accountability Act - US patient data privacy law
ICD-10	International Classification of Diseases - standardized diagnosis coding system
IQR	Interquartile Range ($Q3 - Q1$) - used for outlier detection
LIME	Local Interpretable Model-agnostic Explanations - explains individual predictions
LSTM	Long Short-Term Memory - neural network for sequential/time-series data
MIMIC	Medical Information Mart for Intensive Care - public ICU research database
Overfitting	Model memorizes training data but fails on new data
Pharmacogenomics	Using genetic data to personalize drug therapy

Polypharmacy	Taking 5+ medications simultaneously. Increases ADR risk
Precision	Of all positive predictions, how many were correct? $TP/(TP+FP)$
Recall	Of all actual positives, how many were detected? $TP/(TP+FN)$
ReLU	Rectified Linear Unit - activation function: $\max(0, x)$
SaMD	Software as a Medical Device - FDA classification for clinical AI tools
SHAP	SHapley Additive exPlanations - explains feature contributions to predictions
Sigmoid	Activation function that squashes output to $[0, 1]$ for probability
SMOTE	Synthetic Minority Over-sampling Technique - creates synthetic minority samples
Stratify	Ensuring train/test splits maintain the same class ratio as original data
XGBoost	eXtreme Gradient Boosting - fast, accurate ensemble method for tabular data

Project Complete

GSK Medicine Outcome Prediction System

UC-01: Clinical Documentation Case Study

-
- > 6 ML/DL Models: Decision Tree, Random Forest, XGBoost, Neural Network, LSTM
 - > Model Explainability: SHAP & LIME for regulatory compliance
 - > SQL Analytics: Clinical cohort queries and drug utilization analysis
 - > Deployment: Flask REST API for real-time predictions
 - > Compliance: HIPAA, GDPR, FDA SaMD considerations
 - > 10 Interview Q&As + Comprehensive Glossary

Technologies: Python | Pandas | Scikit-Learn | XGBoost | TensorFlow/Keras | SHAP | Flask